

# Link Prediction with Node Embedding Models

Peter Dolog

[dolog@cs.aau.dk](mailto:dolog@cs.aau.dk)

<http://people.cs.aau.dk/~dolog>

Slides based on chapter 3 from graph representation learning book.

Some slides from Bo Thiesson or adapted from the teaching material supporting the book “Recommender Systems: An Introduction” by Dietmar Jannach, Markus Zanker, Alexander Felfernig & Gerhard Friedrich (2011) and from UC Irvine course taught by Max Welling.

# Outline

Problems with previous methods

Encoder/Decoder view on embeddings methods

Matrix Factorization approaches

- Inner product based methods
- Laplacian eigen maps based methods

Random Walk approaches

- Standard random walk methods: DeepWalk
- Biased random walk methods: Node2Vec

# Link Prediction

If the nodes are (highly) **SIMILAR**, there should be a link between them

We are talking about learning similarity from a graph

**Task:** Define a **similarity measure**  $S(v;w)$  between nodes based on graph structure

**Last week:**

- local structural similarity measures and SimRank (global, based on random walk)
- No learning

# Problems

Computing pairwise similarities and complete random walks can be computationally difficult for large graphs and all nodes

Spamming is a problem

Simple changes can affect the results

Coverage especially for local measures poor

...

# Low dimensional space

**By dimensionality reduction, mapping to a low dimensional space**

**Basic idea:**

- Trade more complex offline model building for faster online prediction generation

**Several models exist**

# General Idea

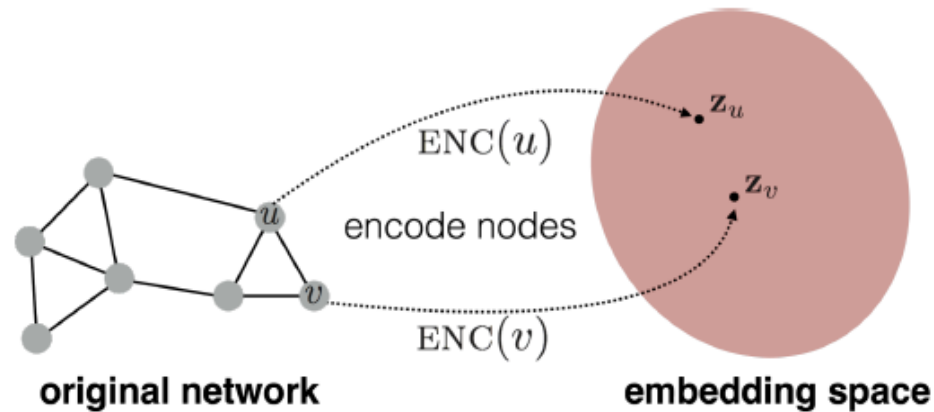


Figure 3.1: Illustration of the node embedding problem. Our goal is to learn an encoder ( $ENC$ ), which maps nodes to a low-dimensional embedding space. These embeddings are optimized so that distances in the embedding space reflect the relative positions of the nodes in the original graph.

(C) William L. Hamilton: Graph Representation Learning

# Encoder/Decoder

Encoding => Learning

Decoding => Prediction

For many prediction tasks incl. link prediction

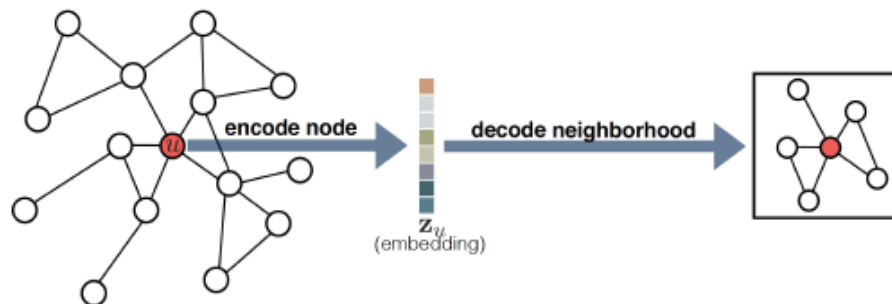


Figure 3.2: Overview of the encoder-decoder approach. The encoder maps the node  $u$  to a low-dimensional embedding  $\mathbf{z}_u$ . The decoder then uses  $\mathbf{z}_u$  to reconstruct  $u$ 's local neighborhood information.

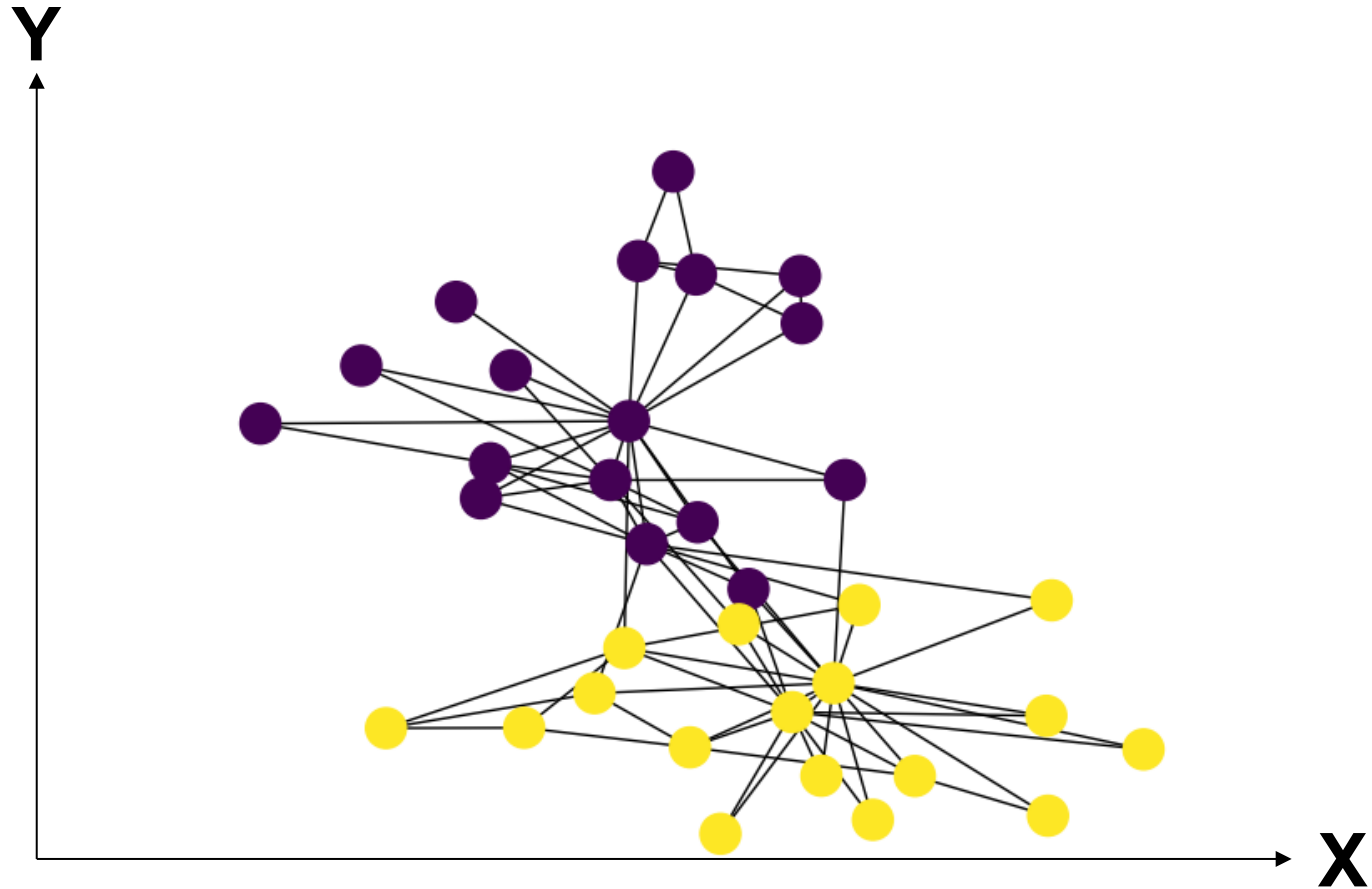
(C) William L. Hamilton: Graph Representation Learning

## Basic Idea

Consider a two-dimensional space given by a drawing canvas  
Draw a graph according to selected drawing algorithm  
Observe and record coordinates of nodes in that space  
Use that as an embeddings



## Illustration of Idea



**Drawn from zachary by `ntx.spring_layout()`**

# Coordinates

```
[[-0.08608312 0.35674596]
[-0.10669304 0.20477623]
[-0.06612635 0.03965846]
[-0.2406135 0.2481614 ]
[ 0.10655595 0.60888028]
[-0.0755657 0.76935902]
[-0.01136205 0.73449742]
[-0.25088574 0.15759357]
[ 0.0472217 -0.07626688]
[ 0.38513712 -0.10531281]
[ 0.10468862 0.73088794]
[-0.27860864 0.66405851]
[-0.4966925 0.35019105]
[-0.00950762 0.09581953]
[ 0.18578961 -0.67118183]
[ 0.12844744 -0.55763504]
[-0.036615 1. ]
[-0.38425909 0.49969972]
[ 0.4874534 -0.4897098 ]
[ 0.15477792 0.20492753]
[ 0.01593903 -0.70716658]
[-0.21774876 0.48734977]
[ 0.37908922 -0.40309779]
[-0.01793822 -0.48426817]
[-0.3567273 -0.43493225]
[-0.20280292 -0.43401062]
[ 0.38093191 -0.66064669]
[-0.12433752 -0.34481378]
[ 0.17073146 -0.11723207]
[ 0.23263402 -0.55833376]
[ 0.03637997 -0.16725268]
[-0.09110846 -0.2283291 ]
[ 0.09586958 -0.39352437]
[ 0.14202861 -0.31889215]]
```

# Formalization

## Encoder:

- Generally:  $ENC: \mathcal{V} \rightarrow \mathbb{R}^d$
- Specifically in node embeddings:  $ENC(v) = \mathbf{Z}[v]$   
Where  $\mathbf{Z} \in \mathbb{R}^{|\mathcal{V}| \times d}$

## Decoder:

- Generally:  $DEC: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^+$
- Specifically in reconstruction of the relationships:  
$$DEC(ENC(u), ENC(v)) = DEC(\mathbf{z}_u, \mathbf{z}_v) \approx \mathbf{S}[u, v]$$

# Learning

We want to minimize the reconstruction loss, since we want to reconstruct the links

As a side effect of this we will also predict missing links

$$\mathcal{L} = \sum_{(u,v) \in \mathcal{D}} \ell(DEC(\mathbf{z}_u, \mathbf{z}_v), \mathbf{S}[u, v])$$

Where  $\ell: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$  is a loss function measuring some kind of a difference between decoded and estimated similarity;  $\mathcal{D}$  are training node pairs

To define: loss function, similarity, DEC function

# ***Matrix Factorization / Inner Product Methods / Dimensionality Reduction***

# Dimensionality reduction

## Basic idea:

- Trade more complex offline model building for faster online prediction generation

## Singular Value Decomposition for dimensionality reduction of rating matrices

- Captures important factors/aspects and their weights in the data
- Factors can be genre, actors but also non-understandable ones
- Assumption that  $k$  dimensions capture the signals and filter out noise ( $K = 20$  to  $100$ )

**Approach also popular in IR (Latent Semantic Indexing), data compression,...**

# Formally

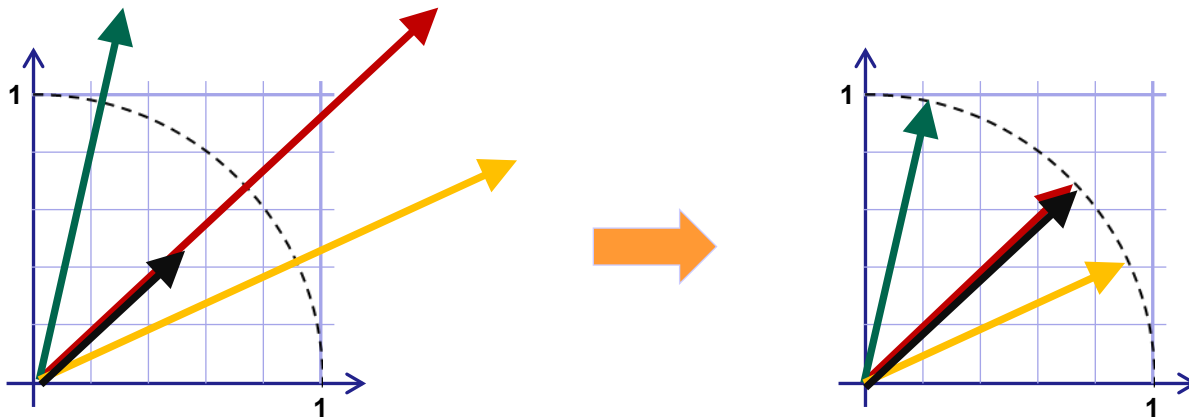
**Decoder / Predictor (dot product):**  $DEC(\mathbf{z}_u, \mathbf{z}_v) = \mathbf{z}_u^T \mathbf{z}_v$

**Loss function:**  $\mathcal{L} = \sum_{(u,v) \in \mathcal{D}} ||DEC(\mathbf{z}_u, \mathbf{z}_v) - \mathbf{S}[u, v]||_2^2$   
 $\mathcal{L} \approx ||\mathbf{Z}\mathbf{Z}^T - \mathbf{S}||_2^2$

## Similarity:

- can differ in different approaches
- Simplest approach:  $\mathbf{S} \triangleq \mathbf{A}$  using adjacency matrix directly
- By fixing this, we can apply matrix factorization approach such as SVD
- In the following I will use recommender systems example where instead of  $\mathbf{A}$  I will talk about  $\mathbf{R}$  and instead of binary representation we will have integer weights. Further, the  $\mathbf{R}$  will represent bipartite graph, i.e. it will not be squared

# Intuition behind Dot Product – Cosine similarity (normalized)



$$\cos(q, d) = \frac{q \cdot d}{|q||d|} = \frac{q}{|q|} \cdot \frac{d}{|d|} = \hat{q} \cdot \hat{d}$$

$$= \frac{\sum_{i=1}^N q_i d_i}{\sqrt{\sum_{i=1}^N q_i^2} \sqrt{\sum_{i=1}^N d_i^2}} = \sum_{i=1}^N \hat{q}_i \hat{d}_i = \cos(\hat{q}, \hat{d})$$

| $\hat{q}$      | $\hat{d}$      |
|----------------|----------------|
| -              | -              |
| -              | $\hat{d}_2$    |
| -              | -              |
| -              | -              |
| $\hat{q}_5$    | -              |
| -              | -              |
| -              | $\hat{d}_7$    |
| -              | -              |
| -              | -              |
| -              | -              |
| -              | $\hat{d}_{11}$ |
| -              | $\hat{d}_{12}$ |
| $\hat{q}_{13}$ | $\hat{d}_{13}$ |
| -              | -              |
| -              | -              |
| $\hat{q}_{16}$ | $\hat{d}_{16}$ |
| -              | $\hat{d}_{17}$ |
| -              | -              |



# Matrix factorization

- Informally, the SVD theorem (Golub and Kahan 1965) states that a given matrix  $M$  can be decomposed into a product of three matrices as follows

$$R = U \times \Sigma \times V^T$$

- where  $U$  and  $V$  are called *left* and *right singular vectors* and the values of the *diagonal of  $\Sigma$*  are called the *singular values*

## We can approximate the full matrix $R$

- by observing only the most important features – those with the largest singular values

## In the example (in two slides)

- we calculate  $U$ ,  $V$ , and  $\Sigma$  (with the help of some linear algebra software) but retain only the two most important features by taking only the first two columns of  $U$  and  $V^T$

## Notice...

Sometimes

$$R = \boxed{\dots r_{up} \dots}$$

But mostly (we take out the user bias)

$$R = \boxed{\dots r_{up} - \bar{r}_u \dots}$$

## Example of SVD-based “recommendation”

- **SVD:**  $R_k = U_k \times \Sigma_k \times V_k^T$

| $U_k$ | Dim1  | Dim2  |
|-------|-------|-------|
| Alice | 0.47  | -0.30 |
| Bob   | -0.44 | 0.23  |
| Mary  | 0.70  | -0.06 |
| Sue   | 0.31  | 0.93  |

| $V_k^T$ | Terminator | Die Hard | Twins | Eat Pray Love | Pretty Woman |
|---------|------------|----------|-------|---------------|--------------|
| Dim1    | -0.44      | -0.57    | 0.06  | 0.38          | 0.57         |
| Dim2    | 0.58       | -0.66    | 0.26  | 0.18          | -0.36        |

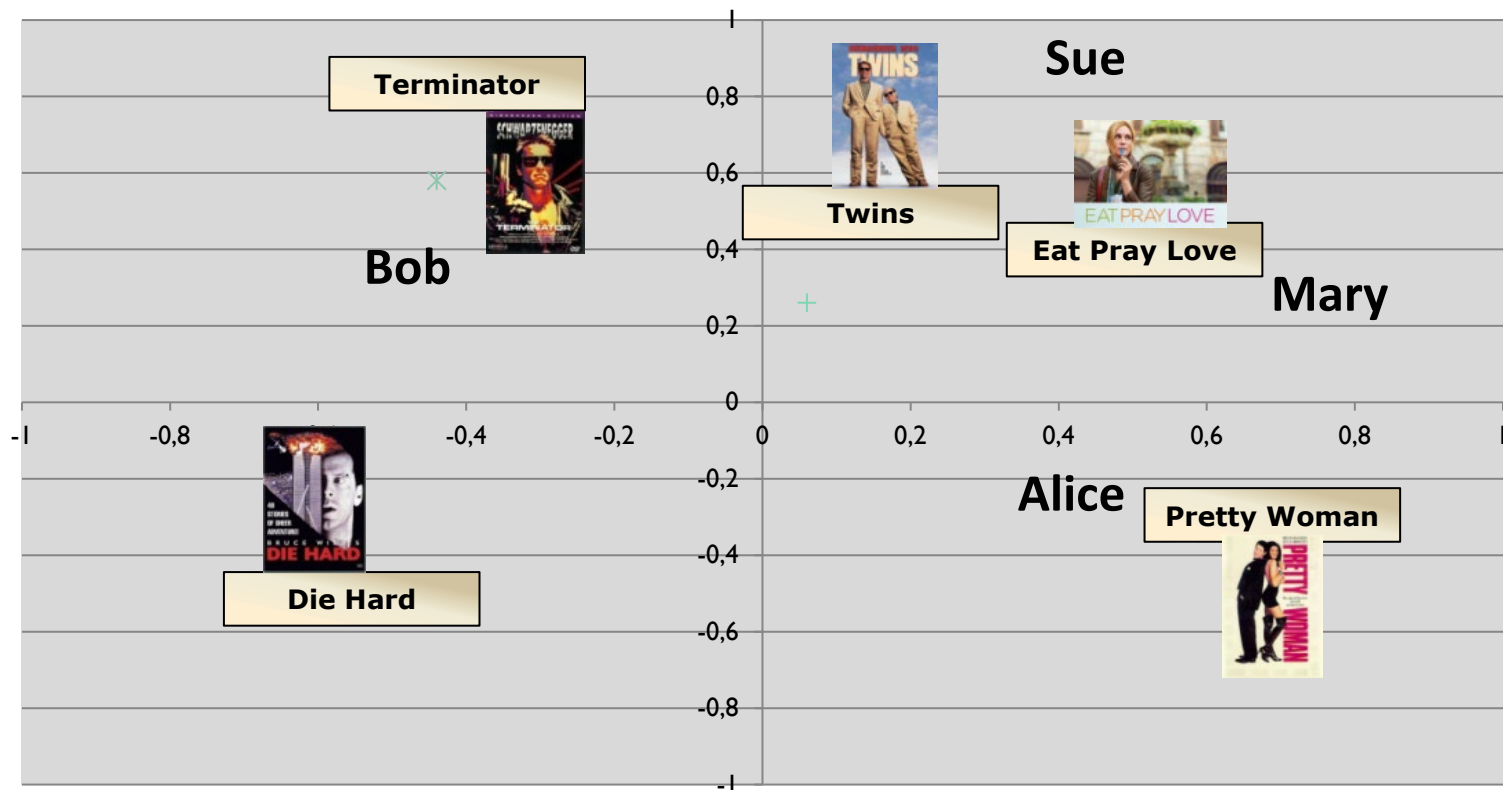
- **Prediction:**  $\hat{r}_{ui} = \bar{r}_u + U_k(\text{Alice}) \times \Sigma_k \times V_k^T(\text{EPL})$   
 $= 3 + 0.84 = 3.84$

| $\Sigma_k$ | Dim1 | Dim2 |
|------------|------|------|
| Dim1       | 5.63 | 0    |
| Dim2       | 0    | 3.23 |

- **Notice:** No magic here (Alice has already rated EPL)
  - Pure dimensionality reduction just takes out the “noise”

# The “latent factor space”

Projection of  $U$  and  $V^T$  in the 2 dimensional space  $(U_2, V_2^T)$



## Now it becomes more interesting...

- **Large scale**
- **Missing ratings (sparse rating matrix)**

# "Funk-SVD" and the Netflix prize

(S. Funk, 2006)

## Netflix announced a million dollar prize

- Goal:
  - Beat their own "Cinematch" system by 10 percent
  - Measured in terms of the Root Mean Squared Error
- Effect:
  - Stimulated lots of research



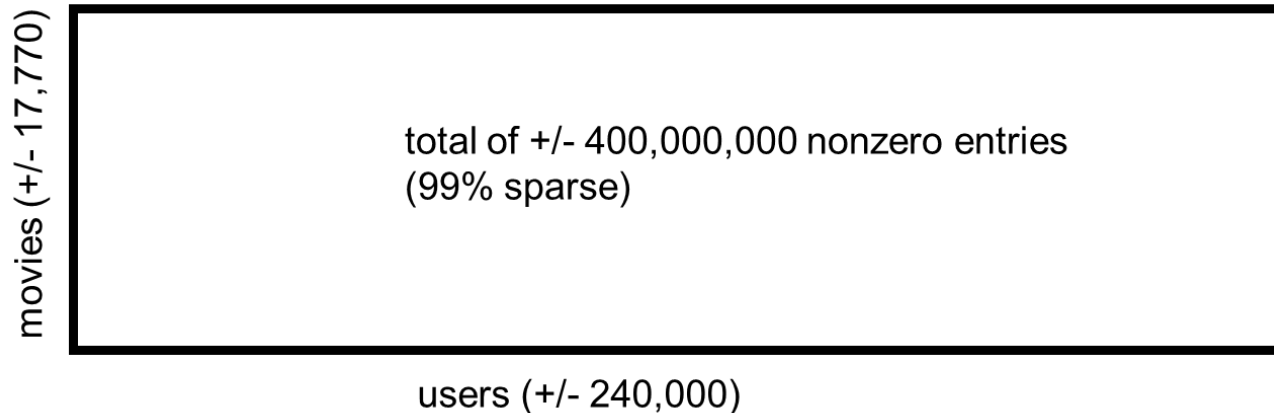
Simon Funk

## Idea of SVD and matrix factorization picked up again

- S. Funk (pen name)
  - **Large scale:**
    - Use fast gradient descent optimization
  - Capable of SVD with **missing ratings**
  - <http://sifter.org/~simon/journal/20061211.html>



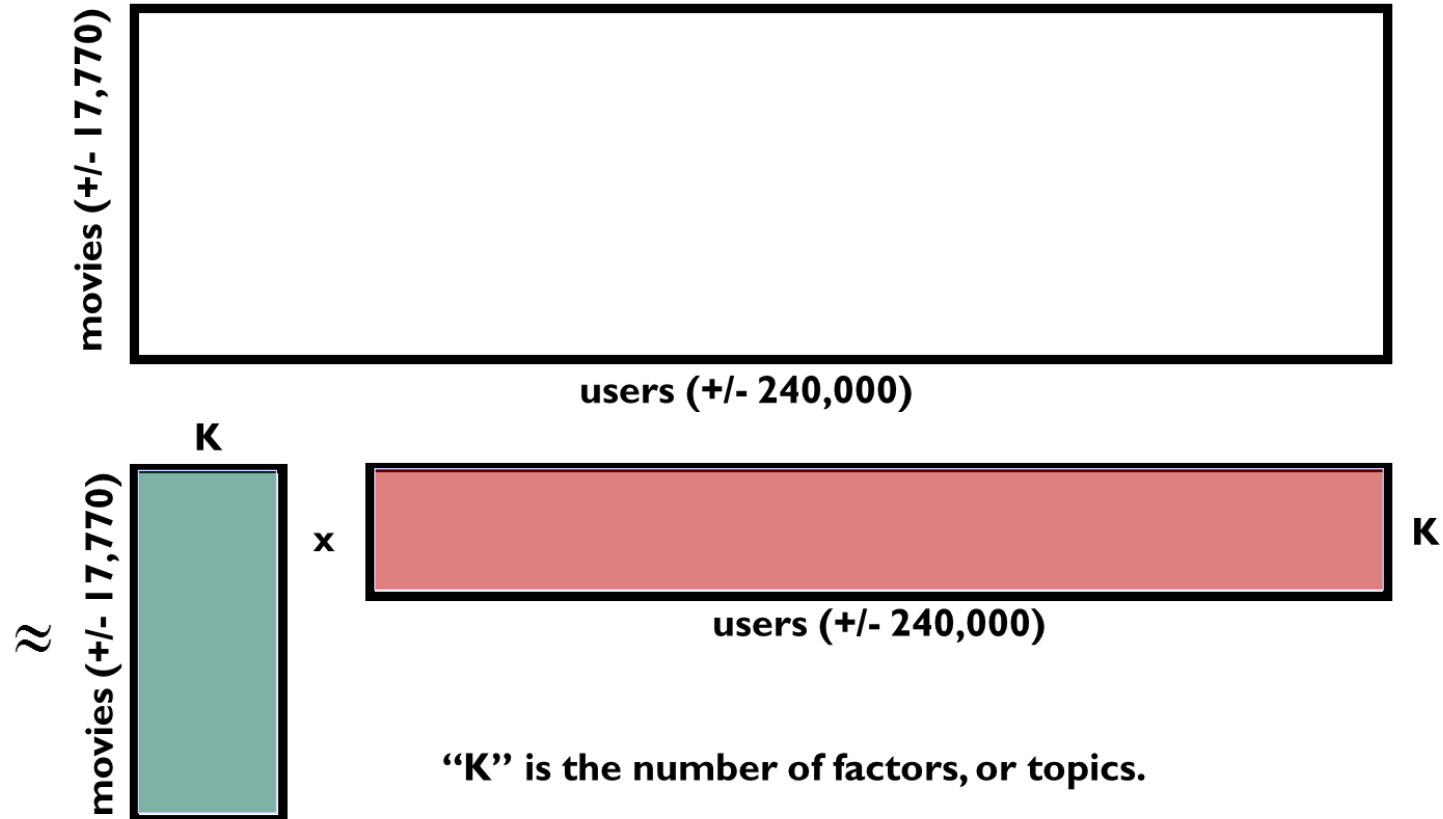
## Remember the Netflix user-item matrix



- We are going to “squeeze out” the noisy, un-informative information from the matrix.
- In doing so, the algorithm will learn to only retain the most valuable information for predicting the ratings in the data matrix.
- This can then be used to more reliably predict the entries that were not rated.

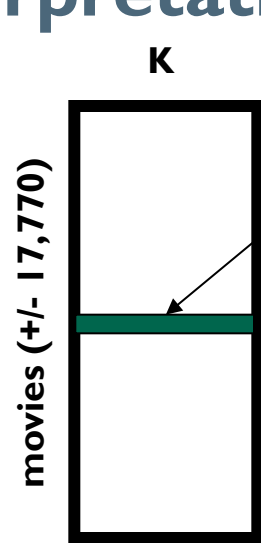
## Squeezing out Information

$$R = U\Sigma V^T = U\Sigma^{1/2}\Sigma^{1/2}V^T = AB$$



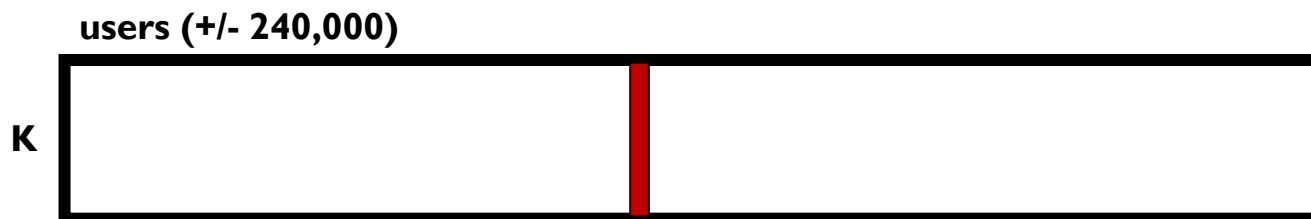


# Interpretation



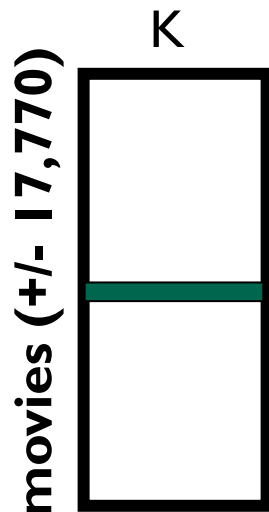
star-wars = [10,3,-4,-1,0.01]

- Before, each movie was characterized by a signature over 240,000 user-ratings.
- Now, each movie is represented by a signature of K “movie-genre” values (or topics, or factors)
- Movies that are similar are expected to have similar values for their movie genres.

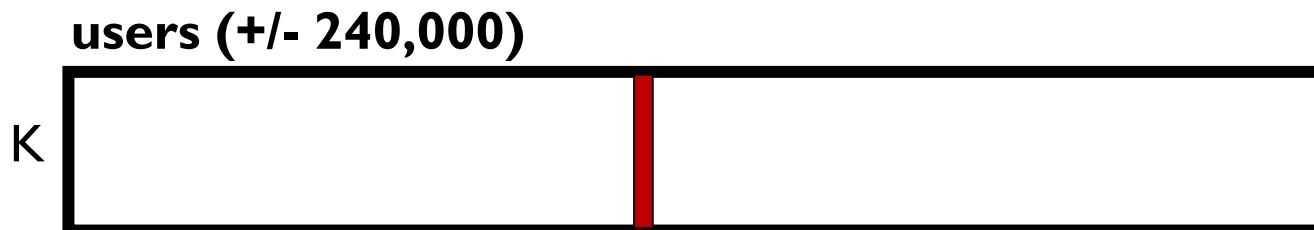


- Before, users were characterized by their ratings over all movies.
- Now, each user is represented by his/her values over movie-genres.
- Users that are similar have similar values for their movie-genres.

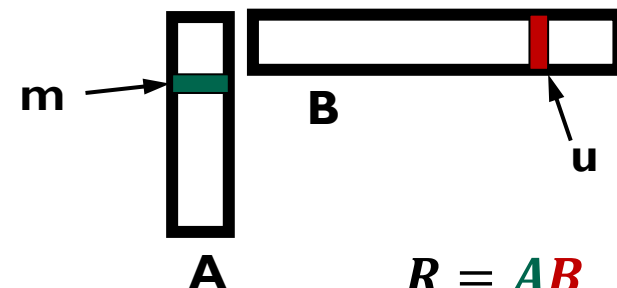
# Dual Interpretation



- Interestingly, we can also interpret the factors/topics as user-communities.
- Each user is represented by its “participation” in K communities.
- Each movie is represented by the ratings of these communities for the movie.
- Pick what you like! I’ll call them topics or factors from now on.



# The Algorithm



- In an equation, the squeezing boils down to:

$$\hat{R}_{mu} \approx \sum_{k=1}^K A_{mk} B_{ku}$$

- We want to find A,B such that we can reproduce the observed ratings as best as we can, given that we are only given K topics.
- To do so, we minimize the squared error (Frobenius norm):

$$Error = \|R - \hat{R}\|_F^2 = \|R - AB\|_F^2 = \sum_{m=1}^M \sum_{u=1}^U \left( R_{mu} - \sum_{k=1}^K A_{mk} B_{ku} \right)^2$$

Note that the error here is computed in reverse order than in the theory part  
Since the squared version is given, the value will be the same

# Prediction – **THE MAGIC**

- **We train  $A, B$  to:**
  - minimize the error for the observed ratings only, and
  - ignore all the non-rated movie-user pairs.
- **But here come the **magic**:**
  - after learning  $A, B$ , the product  $AB$  will have filled-in all the values for the non-rated movie-user pairs for you!
  - It has implicitly used the information from similar users and similar movies to generate ratings for movies that weren't there before.
- **This is what “learning” is:**
  - we can predict things from what we haven't seen before by looking at old data.

# Algorithm – Gradient descent

$$Error = \|R - AB\|_F^2$$

- We want to minimize the Error. The gradient will point in the direction of largest increase

$$\begin{aligned}\frac{dError}{dA} &= -(R - AB)B^T = -\sum_u \left( R_{mu} - \sum_i A_{mi}B_{iu} \right) B_{ku} \quad \forall k \\ \frac{dError}{dB} &= -A^T(R - AB) = -\sum_m A_{mk} \left( R_{mu} - \sum_i A_{mi}B_{iu} \right) \quad \forall k\end{aligned}$$

- So let's go in the opposite direction

$$A \leftarrow A - \eta \frac{dError}{dA} \qquad B \leftarrow B - \eta \frac{dError}{dB}$$

- But wait!**

- how do we ignore the non-observed entries?
- for Netflix, this will actually be very slow, how do we scale up?

## Better algorithm – Stochastic gradient descent

- Pick a single observed movie-user pair rating at random:  $R_{mu}$
- Ignore the sums over  $u, m$  in the exact gradients, and do an update:

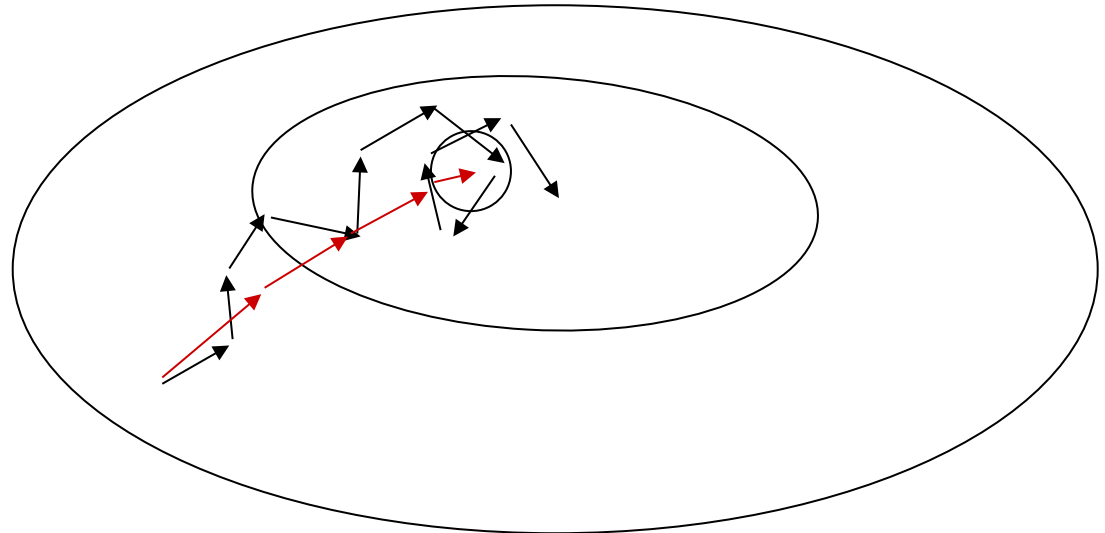
$$A_{mk} \leftarrow A_{mk} + \eta \sum_u \left( R_{mu} - \sum_i A_{mi} B_{iu} \right) B_{ku} \quad \forall k$$

“S. Funk”:  $\eta = 0.001$   
good value for  
Netflix data

$$B_{ku} \leftarrow B_{ku} + \eta \sum_m A_{mk} \left( R_{mu} - \sum_i A_{mi} B_{iu} \right) \quad \forall k$$

- The trick is that although we don’t follow the exact gradient, on average we do move in the correct direction.

# Stochastic Gradient Descent



→ stochastic updates

→ full updates (averaged over all data-items)

- Stochastic gradient descent does not converge to the minimum, but “dances” around it.
- To get to the minimum, one needs to decrease the step-size as one get closer to the minimum.
- Alternatively, one can obtain a few samples and average predictions over them

# Weight-Decay (Regularization)

$$\text{Regularized Error} = \|R - AB\|_F^2 + \lambda(\|A\|_F^2 + \|B\|_F^2)$$

- Often it is good to make sure the values in A,B do not grow too big.
- We can make that happen by adding weight decay terms which will keep them small.
- The simplest weight decay results in:

$$A_{mk} \leftarrow A_{mk} + \eta \left( R_{mu} - \sum_i A_{mi} B_{iu} \right) B_{ku} - \lambda A_{mk} \quad \forall k$$

$$B_{ku} \leftarrow B_{ku} + \eta A_{mk} \left( R_{mu} - \sum_i A_{mi} B_{iu} \right) - \lambda B_{ku} \quad \forall k$$



# Pre-Processing

## We are almost ready for implementation

- We can make a head-start if we first remove some “obvious” structure from the data, so the algorithm doesn’t have to search for it.
- In particular, you can subtract the user and movie means where you ignore missing entries.

$$R_{mu} \leftarrow R_{mu} - \frac{1}{U_m} \sum_s R_{ms} - \frac{1}{M_u} \sum_r R_{ru} + \frac{1}{N} \sum_s \sum_r R_{sr}$$

- $U_m$ : total number of observed ratings for movie  $m$
- $M_u$ : total number of observed ratings for user  $u$
- $N$ : total number of observed movie-user pairs.

# Final prediction model

- Do the matrix factorization for the residual

$$\hat{R}_{mu} \approx \sum_{k=1}^K A_{mk} B_{ku}$$

- **For the predictions: Remember to add in the “obvious” structure that was removed before the matrix factorization**

$$\hat{R}_{mu} \leftarrow \hat{R}_{mu} + \frac{1}{U_m} \sum_s R_{ms} + \frac{1}{M_u} \sum_r R_{ru} - \frac{1}{N} \sum_s \sum_r R_{sr}$$

# Changing the DEC and Loss Functions

**Decoder / Predictor (squared Euclidean distance):**

$$DEC(\mathbf{z}_u, \mathbf{z}_v) = \|\mathbf{z}_u - \mathbf{z}_v\|_2^2$$

**Loss function:**  $\mathcal{L} = \sum_{(u,v) \in \mathcal{D}} DEC(\mathbf{z}_u, \mathbf{z}_v) \cdot S[u, v]$

**Similarity:**

- $S \triangleq A$  using adjacency matrix directly, but can be also defined with weights – e.g. similarity scores
- Here we can apply eigen value and eigen vector decomposition with Laplacian matrix derived from  $A$

# Some math...

## Eigenvalue & eigenvector

A (non-zero) **vector**  $v$  and **scalar**  $\lambda$  are, respectively, an **eigenvector** and corresponding **eigenvalue** of square matrix  $A$  if and only if

$$Av = \lambda v$$

## Eigen decomposition

Let  $A$  be a symmetric square matrix. An **eigen-decomposition** of  $A$  is a factorization

$$A = V^T \Lambda V,$$

where  $V$  is a square matrix with linear independent eigenvectors of  $A$  and  $\Lambda$  is a diagonal matrix whose diagonal elements are the corresponding eigenvalues.

# Laplacian

Symetric:  $\mathbf{L} = \mathbf{L}^T$

Positive semi-definite:  $\mathbf{x}^T \mathbf{L} \mathbf{x} \geq 0, \forall \mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}$

The following vector identity holds:

$$\mathbf{x}^T \mathbf{L} \mathbf{x} = \frac{1}{2} \sum_{u \in \mathcal{V}} \sum_{v \in \mathcal{V}} \mathbf{A}[u, v] (\mathbf{x}[u] - \mathbf{x}[v])^2$$

## Laplacian cntd.

Symetric:  $\mathbf{L} = \mathbf{L}^T$

Positive semi-denite:  $\mathbf{x}^T \mathbf{L} \mathbf{x} \geq 0, \forall \mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}$

The following vector identity holds:

$$\mathbf{x}^T \mathbf{L} \mathbf{x} = \frac{1}{2} \sum_{u \in \mathcal{V}} \sum_{v \in \mathcal{V}} \mathbf{A}[u, v] (\mathbf{x}[u] - \mathbf{x}[v])^2$$

Notice similarity  
with eigen  
decomposition

Notice similarity  
with Loss  
function

## Laplacian cntd.

Symetric:  $\mathbf{L} = \mathbf{L}^T$

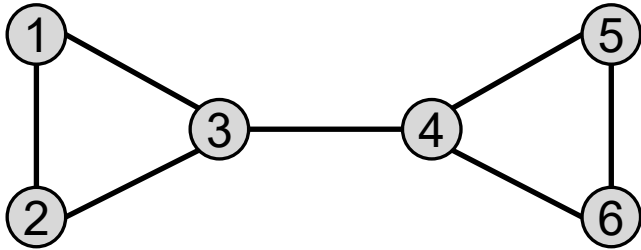
Positive semi-definite:  $\mathbf{x}^T \mathbf{L} \mathbf{x} \geq 0, \forall \mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}$

The following vector identity holds:

$$\mathbf{x}^T \mathbf{L} \mathbf{x} = \frac{1}{2} \sum_{u \in \mathcal{V}} \sum_{v \in \mathcal{V}} \mathbf{A}[u, v] (\mathbf{x}[u] - \mathbf{x}[v])^2$$

$\mathbf{L}$  has  $|\mathcal{V}|$  non-negative eigen values:  $0 = \lambda_{|\mathcal{V}|} \leq \lambda_{|\mathcal{V}|-1} \leq \dots \leq \lambda_1$

## Laplacian cntd.



| <b>A</b> | 1        | 2        | 3        | 4        | 5        | 6        |
|----------|----------|----------|----------|----------|----------|----------|
| 1        | 0        | $w_{12}$ | $w_{13}$ | 0        | 0        | 0        |
| 2        | $w_{21}$ | 0        | $w_{23}$ | 0        | 0        | 0        |
| 3        | $w_{31}$ | $w_{32}$ | 0        | $w_{34}$ | 0        | 0        |
| 4        | 0        | 0        | $w_{43}$ | 0        | $w_{45}$ | $w_{46}$ |
| 5        | 0        | 0        | 0        | $w_{54}$ | 0        | $w_{56}$ |
| 6        | 0        | 0        | 0        | $w_{64}$ | $w_{65}$ | 0        |

**Weighted degree:**

$$d_i = \sum_j w_{ij}$$

Un-normalized **Laplacian:**

$$L = D - A$$

| <b>D</b> | 1     | 2     | 3     | 4     | 5     | 6     |
|----------|-------|-------|-------|-------|-------|-------|
| 1        | $d_1$ | 0     | 0     | 0     | 0     | 0     |
| 2        | 0     | $d_2$ | 0     | 0     | 0     | 0     |
| 3        | 0     | 0     | $d_3$ | 0     | 0     | 0     |
| 4        | 0     | 0     | 0     | $d_4$ | 0     | 0     |
| 5        | 0     | 0     | 0     | 0     | $d_5$ | 0     |
| 6        | 0     | 0     | 0     | 0     | 0     | $d_6$ |



# Eigen Decomposition

Algebraic as well as approximate/learning approaches exist

# RANDOM WALK BASED APPROACHES

# Idea

Recall in matrix factorization:

$$\mathbf{z}_u^T \mathbf{z}_v \approx \mathbf{S}[u, v] \text{ or}$$
$$\|\mathbf{z}_u - \mathbf{z}_v\|_2^2 \approx \mathbf{S}[u, v]$$

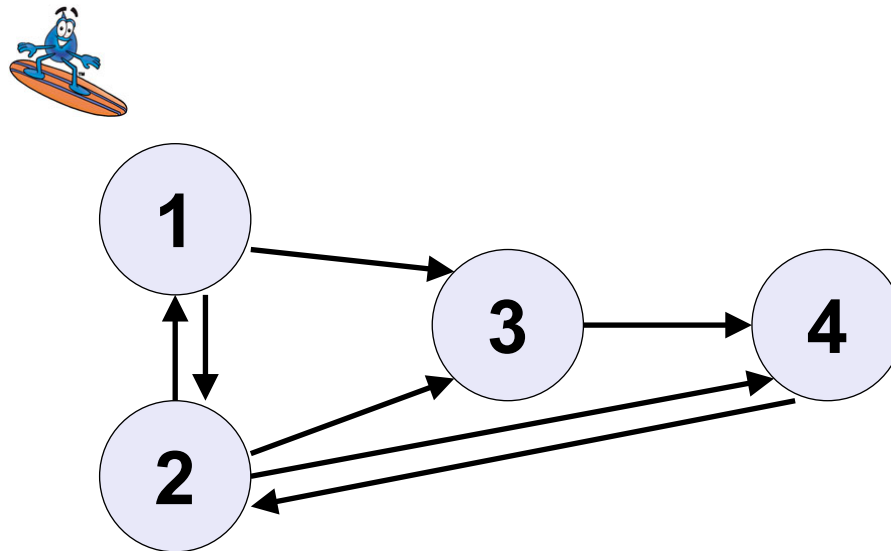
Lets instead:

$$\mathbf{z}_u^T \mathbf{z}_v \approx$$

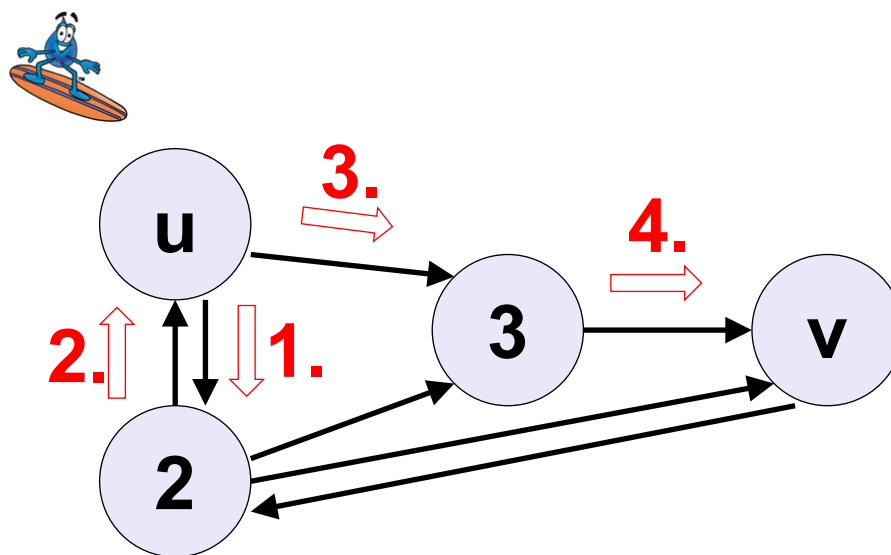
Probability that  $u$  and  $v$  co-occur  
on short random walks over graphs

Similar advantage as using global similarity methods over local similarity methods

# Recall Random Walk



# Recall Random Walk



# Recall Definition of PageRank

(Brin and Page '98)

Consider the following infinite random walk (surf):

- Initially the surfer is at a random page
- At each step, the surfer proceeds
  - to a randomly chosen web page with probability  $\alpha$
  - to a randomly chosen successor of the current page with probability  $1 - \alpha$

The PageRank of a page  $p$  is the fraction of steps the surfer spends at  $p$  in the limit. (A score between 0 and 1)

Random walk provides some statistics over random walk which can be used for estimating “neighbourhood overlap”

# Task

Estimate the probability:

$$P_R(v|u)$$

Probability that  $v$  appears on (short) random walks starting at  $u$   
For all nodes

# Random Walk Embeddings

1. Estimate probability of visiting node  $v$  on a random walk starting from node  $u$  using some random walk strategy:  
 $P_R(v|u)$
2. Optimize embeddings to encode such statistics (dot product – angle is proportional to the probability):  $\mathbf{z}_u^T \mathbf{z}_v \propto P_R(v|u)$



# Formally

**Decoder / Predictor (dot product):**  $DEC(\mathbf{z}_u, \mathbf{z}_v) = \mathbf{z}_u^T \mathbf{z}_v$

## Loss function:

With max likelihood objective:  $\mathcal{L} = \arg \max_z \sum_{u \in V} \log P(N_R(u) | \mathbf{z}_u)$

- Optimize embeddings such that given node  $u$ , predict the neighbourhood

With min likelihood objective:  $\arg \min_z \mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v | \mathbf{z}_u))$

- Optimize embeddings to minimize negative log likelihood of random walk neighborhood

Where  $N_R(u)$  is the neighbourhood of node  $u$  computed by some random walk strategy  $R$

## Similarity:

$$P_R(v|u)$$

## Probability – given the minimization version

Using SoftMax:

$$P(v|\mathbf{z}_u) = \frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}$$

Still dot product, but maximizing it and normalize w.r.t sum of all dot products to give a probability

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log\left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}\right)$$

## Need for Optimization

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log\left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}\right)$$

Nested sums over all nodes are too expensive

## Need for Optimization

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log\left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}\right)$$

Nested sums over all nodes are too expensive

Using sigmoid and negative sampling

$$\mathcal{L} = \sum_{(u,v) \in \mathcal{D}} -\log\left(\sigma(\mathbf{z}_u^T \mathbf{z}_v)\right) - \gamma \sum_{i=1}^k \log(\sigma(-\mathbf{z}_u^T \mathbf{z}_k))$$

Where we sample  $k$  negative samples from a distribution over set of nodes

Reports  $k = 5 - 20$  balancing robustness and negative bias

Can be optimized by (stochastic) gradient descent

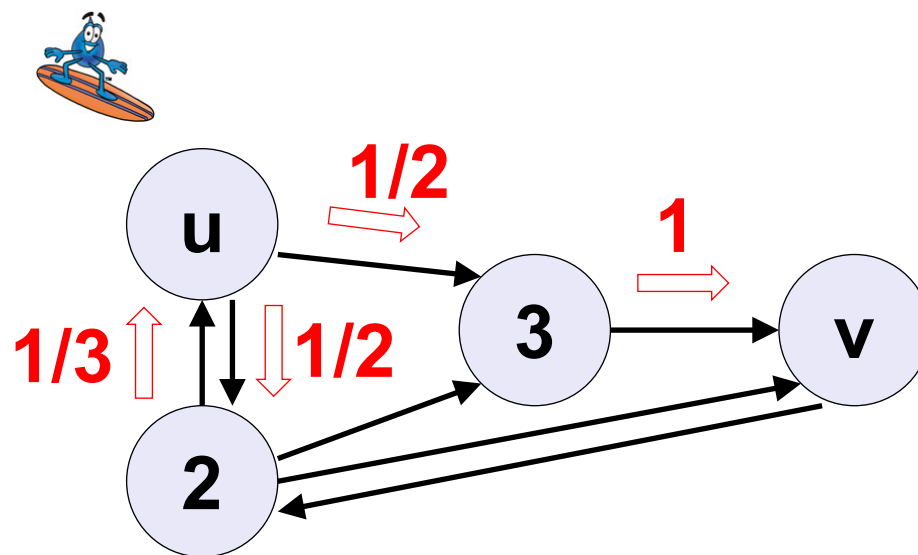
# Strategies for Walks

Standard Random Walk: **DeepWalk** method

Biased Random Walk:

- in each step of random walk we behave according to a bias to return back or to move away
- Done by **Node2Vec**

# Standard Random Walk



# Biased Random Walk

Locally biasing the next steps

Local view is ensured by Breadth-First Search (BFS)

Global view is ensured by Depth-First Search (DFS)

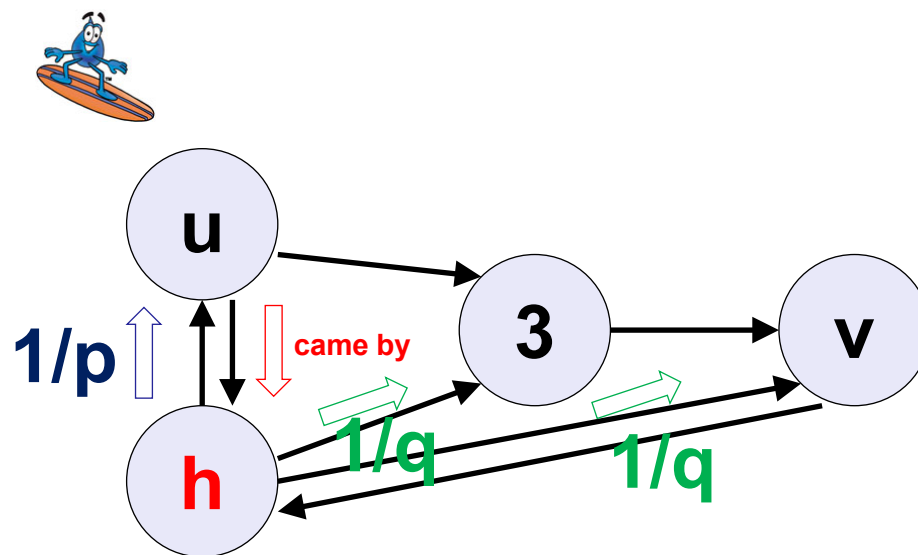
Balancing the two views by parameters  $p$  and  $q$

$p$  is for return / BFS

$q$  is for walking away / DFS

$p$  and  $q$  serve as normalization parameters for the transition probabilities

# Biased Random Walk





# Summary

- We can transform from raw data space to **lower dimensional space**
- Simulating **similarities in such low dimensional space**
- **Key** is to choose **right similarity** for the task at hand
- **Different methods exist** even beyond the ones presented here
- Methods presented are so called **shallow embedding** methods
- We called them **node embeddings** methods since we map each node one by one to the low dimensional space
- **Easy to use**
- **Limitations:**
  - There are no shared parameters between nodes
  - No node features used in encoding
  - They know only nodes which have been there during training